Frontend Developer Coding Exercise

Build a Product Detail Page

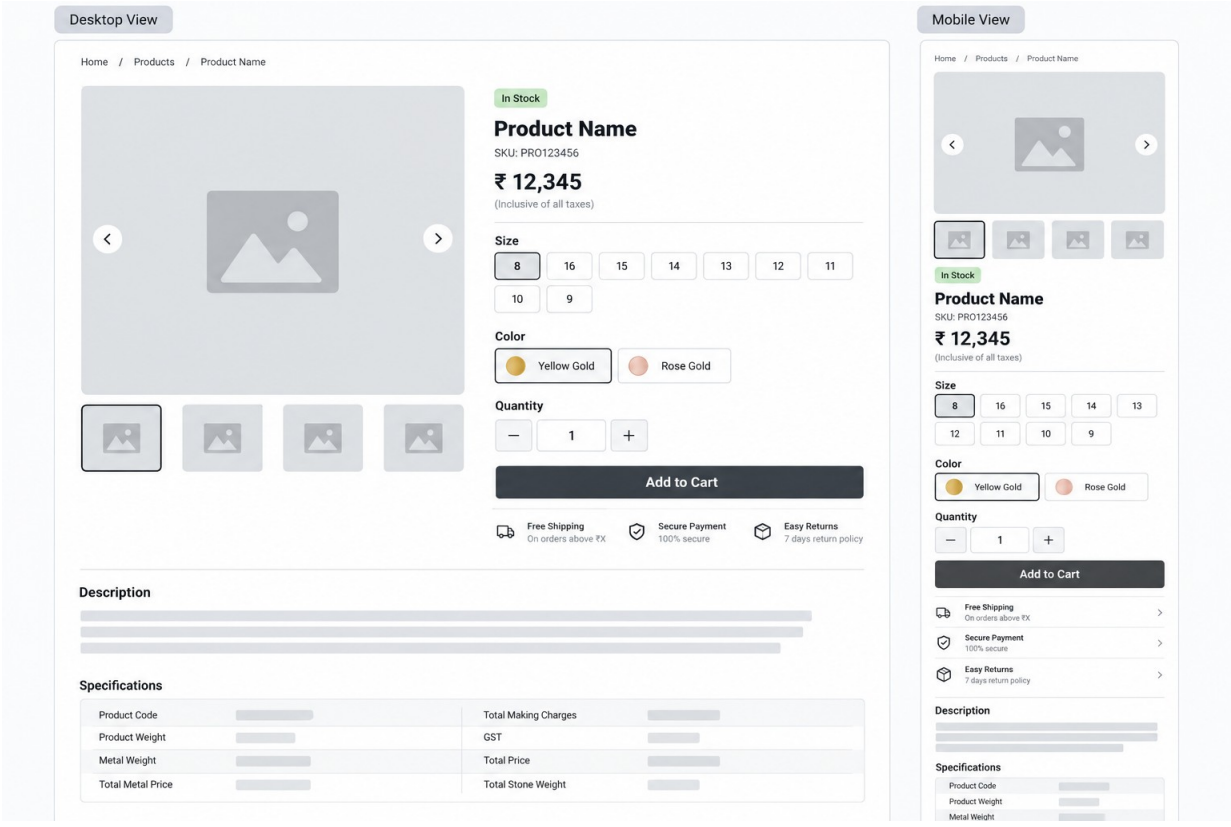
1. Objective

Build a responsive, functional product detail page for an e-commerce application using the provided product API. The page should include product images, product information, variant selection, pricing, quantity selection, availability, and product specifications.

The website header and footer are excluded. Focus exclusively on the product detail page.

2. Wireframe

Use the wireframe below as the structural guide for the expected desktop and mobile layouts. It is a guide, not a pixel-perfect design; use the actual API response to determine the product content, available variants, images, pricing, and specifications.



3. Technical Requirements

- React or Next.js.

- TypeScript is preferred.
- Tailwind CSS or another styling solution.
- Any suitable icon library.
- Native Fetch API or Axios for API requests.

The application should be well-structured, responsive, and easy to maintain.

4. API Integration

Method: GET

Endpoint:

https://b2cqa.bufferbytes.in/customer/products/59bbc2b3-0ce0-45b6-91d6-efd1ecbb012f/

Required request header:

Header	Value
x-vendor-id	a9e3d24d-a2fe-4fbb-830f-91e0fee9f299

Example request:

```
const response = await fetch(
  "https://b2cqa.bufferbytes.in/customer/products/59bbc2b3-0ce0-45b6-91d6-efd1ecbb012f/",
  {
    method: "GET",
    headers: {
      "x-vendor-id": "a9e3d24d-a2fe-4fbb-830f-91e0fee9f299",
    },
  }
);

if (!response.ok) {
  throw new Error("Failed to fetch product details");
}

const product = await response.json();
```

Important: The actual JSON response could not be retrieved while preparing this assignment. The interviewer should paste the verified response below before distributing the task. Candidates must use the actual response rather than a fabricated product object.

API Response:

Response format (illustrative structure only): the exact field names and nesting must be confirmed from the live API response. This example is not the actual API payload.

```
{
  "id": "string",
  "name": "string",
  "sku": "string",
  "description": "string",
```

```
  "price": "number",
  "currency": "string",
  "availability": "string",
  "images": [
    {
      "url": "string",
      "alt": "string"
    }
  ],
  "variants": [
    {
      "id": "string",
      "name": "string",
      "options": {
        "size": "string",
        "color": "string"
      },
      "price": "number",
      "available": "boolean",
      "stock_quantity": "number"
    }
  ],
  "specifications": {
    "product_weight": "string",
    "metal_weight": "string",
    "making_charges": "number",
    "gst": "number",
    "stone_weight": "string"
  }
}
```

Important: This is a field guide for the kind of data the UI may need, not a guaranteed schema. Use the actual response returned by the endpoint; do not assume these names, types, or nesting match the API.

Candidates should identify the relevant fields in the response and use them to populate the UI. Do not hardcode product information that is available through the API.

5. Functional Requirements

A. Product Image Gallery

- Display the primary product image prominently and additional images as thumbnails.
- Allow users to select a thumbnail to change the main image.
- Use image URLs provided by the API.
- Maintain image aspect ratio without stretching or distortion.
- Provide a reasonable fallback if an image fails to load.

B. Product Information

Display relevant information available in the API response, including product name, SKU or product code, price, availability status, description, and specifications. Format prices and numerical values appropriately.

C. Variant Selection

Implement interactive variant selection based on the variants available in the API response.

- Display available sizes as selectable buttons; clearly distinguish the selected size and indicate unavailable sizes.
- Display available color options (for example, Yellow Gold or Rose Gold, if present in the API data) and clearly distinguish the selected color.
- Keep the selected size and color in application state.
- Ensure the selected combination represents a valid variant; do not assume every size/color combination is available.
- Update price and availability when the selected variant changes, if variant-specific values are provided by the API.
- Prevent selection of unavailable variants.

D. Quantity Selector

- Default quantity is 1.
- Increment and decrement buttons update the quantity immediately.
- Quantity cannot be reduced below 1.
- Prevent adding an invalid quantity to the cart.

E. Add-to-Cart Interaction

- Validate the selected variant before adding the product.
- Require the user to select necessary options.
- Prevent adding unavailable products or variants.
- Use the selected variant and quantity when handling the action.
- Provide clear feedback when the product is added successfully.

Backend cart integration is not required unless the provided API supports it and you choose to implement it; a functional frontend interaction is sufficient. If the product is sold out, display the appropriate availability message and disable the purchase action.

F. Product Description and Specifications

Display the product description and available specifications below the main product section. Relevant fields may include product code, product weight, metal weight, total price, total metal price, making charges, GST, and stone-related information. Use only fields present in the API response and do not display empty or undefined values.

6. UI and Responsive Design

- Clean, professional e-commerce UI with balanced spacing and typography.
- Large product imagery and clearly visible product pricing.
- Distinct selected and unselected variant states and consistent button styles.
- Loading and error states.
- Responsive layouts for desktop, tablet, and mobile.
- No horizontal overflow on smaller screens.

The header and footer are explicitly excluded from the assignment.

7. API and State Management

Loading: Show a loading indicator or skeleton while fetching product information.

Success: Render the product page using the API response.

Error: Show an appropriate error message if the request fails; avoid rendering broken or incomplete product information.

State management: Manage selected image, selected variant, quantity, and cart feedback using an appropriate React state approach. A global state-management library is not required.

8. Code Quality Expectations

- Component structure and separation of responsibilities.
- Reusable UI components and readable, maintainable code.
- Appropriate TypeScript types, if TypeScript is used.
- Proper API integration and error handling.
- Correct handling of variant combinations and application state.
- Responsive implementation.
- Avoidance of unnecessary duplication and hardcoded product data.

You may use libraries and tools of your choice, but you should be able to explain the implementation and significant technical decisions.

9. Deliverables

1. Complete source code in a Git repository.
2. README with setup and run instructions.
3. A working local application.
4. A deployed preview, if possible.
5. A brief explanation of component structure and important implementation decisions.

The project should be runnable using the instructions provided in the README.

10. Time Limit

Suggested duration: 2–2.5 hours.

We value implementation quality and functional correctness over the number of features completed. If you cannot finish every requirement, prioritize the main product layout, API integration, variant selection, and responsive design.

11. Final Checklist

☐ Product information comes from the provided API.

☐ The required x-vendor-id header is included.

☐ Product images load correctly and thumbnails update the main image.

☐ Size and color selections work correctly.

☐ Invalid variant combinations cannot be selected.

☐ Price and availability reflect the selected variant where applicable.

☐ Quantity cannot be less than one.

☐ Add to Cart validates the selection.

☐ Description and specifications are displayed correctly.

☐ Loading and error states are implemented.

☐ The page works on desktop and mobile.

☐ The project includes setup instructions.

Good luck!